1. Calculate the updated weights for a perceptron after each epoch using Gradient Descent. Use eta= , b= , and initial weights are [0.1,-0.3]. Run 6 epochs of the perceptron learning rule on a diabetic dataset having 10 patients record.

```
import numpy as np

# Step 1: Dummy diabetic dataset (10 patients, 2 features)
X = np.array([
    [2, 3],
    [1, 5],
    [2, 1],
    [3, 2],
    [4, 1],
    [1, 0],
    [0, 1],
    [5, 2],
    [3, 4],
    [2, 4]
])

y = np.array([1, 1, 0, 0, 1, 0, 0, 1, 1, 1])

# Step 2: Initialize parameters
eta = 0.1
weights = np.array([0.1, -0.3])
bias = 0
epochs = 6

# Step 3: Training loop
for epoch in range(epochs):
    for i in range(len(X)):
        linear_output = np.dot(X[i], weights) + bias
        y_pred = 1 if linear_output >= 0 else 0

        error = y[i] - y_pred

        # Update weights and bias
        weights = weights + eta * error * X[i]
        bias = bias + eta * error

    print(f"Epoch {epoch+1}: Weights = {weights}, Bias = {bias}")
```

```
Epoch 1: Weights = [0.4 0.2], Bias = 0.0
Epoch 2: Weights = [2.00000000e-01 8.32667268e-17], Bias = -0.2
Epoch 3: Weights = [3.00000000e-01 8.32667268e-17], Bias = -0.30000000000000004
Epoch 4: Weights = [4.00000000e-01 8.32667268e-17], Bias = -0.4
Epoch 5: Weights = [0.5 0.3], Bias = -0.4
Epoch 6: Weights = [0.4 0.1], Bias = -0.5
```

2. Develop a feedforward network to find Y

```
import numpy as np

# Step 1: Inputs (example values)
X = np.array([2, 3])

# Step 2: Weights and bias
weights = np.array([0.1, -0.3])
bias = 0.2

# Step 3: Sigmoid activation
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Step 4: Feedforward computation
z = np.dot(X, weights) + bias
Y = sigmoid(z)

print("Net input (z):", z)
print("Output (Y):", Y)
```

```
Net input (z): -0.49999999999999994
Output (Y): 0.3775406687981454
```

3. Design and Develop a Sequential Multi-Layer Perceptron (MLP) using the Keras framework to classify patient health outcomes. Evaluate how different activation functions (sigmoid/ReLU) impact the learning process when using Stochastic Gradient Descent (SGD). Collect the percentage of train_testing_split of records in the laboratory.

```
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD

# Customizable Split
split_size = 0.3   # change in exam

# Sample dataset
X = np.random.randint(0,2,(100,4))
y = np.random.randint(0,2,100)

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=split_size)

# Build Model
def create_model(activation_hidden):
    model = Sequential()
    model.add(Dense(8, input_dim=4, activation=activation_hidden))
    model.add(Dense(4, activation=activation_hidden))
    model.add(Dense(1, activation='sigmoid'))

    model.compile(optimizer=SGD(learning_rate=0.01),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])
    return model

# Sigmoid model
model_sigmoid = create_model('sigmoid')
model_sigmoid.fit(X_train,y_train,epochs=10,verbose=0)
print("Sigmoid Accuracy:", model_sigmoid.evaluate(X_test,y_test)[1])

# ReLU model
model_relu = create_model('relu')
model_relu.fit(X_train,y_train,epochs=10,verbose=0)
print("ReLU Accuracy:", model_relu.evaluate(X_test,y_test)[1])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 174ms/step - accuracy: 0.4000 - loss: 0.7353
Sigmoid Accuracy: 0.4000000059604645
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 166ms/step - accuracy: 0.5333 - loss: 0.6886
ReLU Accuracy: 0.5333333611488342
```

4. Deeply analyse an image as input image matrix, feature matrix, trainable parameters, and output. Create a sequential convolution neural network for this purpose. The model contains 8, 16, 32 filters of each size 2X2 in individual layers. Sigmoid is in the output layer and ReLU is used in the hidden layers. The spatial dimensions of the features are reduced with Average Pooling.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, AveragePooling2D, Flatten, Dense

# Sequential CNN Model
model = Sequential()

# Conv Layer 1
model.add(Conv2D(
    filters=8,
    kernel_size=(2, 2),
    activation='relu',
    input_shape=(28, 28, 1)
))
model.add(AveragePooling2D(pool_size=(2, 2)))

# Conv Layer 2
model.add(Conv2D(
    filters=16,
    kernel_size=(2, 2),
    activation='relu'
))
```

```
model.add(AveragePooling2D(pool_size=(2, 2)))

# Conv Layer 3
model.add(Conv2D(
    filters=32,
    kernel_size=(2, 2),
    activation='relu'
))
model.add(AveragePooling2D(pool_size=(2, 2)))

# Fully Connected Layers
model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))

# Model Summary
model.summary()
```

**Model: "sequential_5"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d_6 (Conv2D) | (None, 27, 27, 8) | 40 |
| average_pooling2d_3 (AveragePooling2D) | (None, 13, 13, 8) | 0 |
| conv2d_7 (Conv2D) | (None, 12, 12, 16) | 528 |
| average_pooling2d_4 (AveragePooling2D) | (None, 6, 6, 16) | 0 |
| conv2d_8 (Conv2D) | (None, 5, 5, 32) | 2,080 |
| average_pooling2d_5 (AveragePooling2D) | (None, 2, 2, 32) | 0 |
| flatten_3 (Flatten) | (None, 128) | 0 |
| dense_10 (Dense) | (None, 1) | 129 |

**Total params:** 2,777 (10.85 KB)
**Trainable params:** 2,777 (10.85 KB)
**Non-trainable params:** 0 (0.00 B)

5. Build a sequential Convolution Neural Network model to classify images from a 'dataset'. Add the layers containing 2n and 2X2n filters. Collect the n value from the teacher. The size of the filter may vary layer to layer.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Value of n
n = 8   # Example value

# Build Sequential CNN
model = Sequential()

# Conv Layer 1: 2n filters
model.add(Conv2D(
    filters=2 * n,
    kernel_size=(3, 3),        # filter size can vary
    activation='relu',
    input_shape=(64, 64, 3)    # example image size
))
model.add(MaxPooling2D(pool_size=(2, 2)))

# Conv Layer 2: 2 × (2n) = 4n filters
model.add(Conv2D(
    filters=4 * n,
    kernel_size=(5, 5),        # different filter size
    activation='relu'
))
model.add(MaxPooling2D(pool_size=(2, 2)))

# Fully Connected Layers
model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))  # binary classification

# Model summary
model.summary()
```

```
model.summary()
```

**Model: "sequential_7"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d_11 (Conv2D) | (None, 62, 62, 16) | 448 |
| max_pooling2d_5 (MaxPooling2D) | (None, 31, 31, 16) | 0 |
| conv2d_12 (Conv2D) | (None, 27, 27, 32) | 12,832 |
| max_pooling2d_6 (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| flatten_5 (Flatten) | (None, 5408) | 0 |
| dense_12 (Dense) | (None, 1) | 5,409 |

**Total params:** 18,689 (73.00 KB)
**Trainable params:** 18,689 (73.00 KB)
**Non-trainable params:** 0 (0.00 B)